Smart Parking KRR: asaan guide

Oct 4, 2026 · @Engineers

1. Project kya hai, aur kyun banaya

Seedhi baat: humara project ek "smart" system hai jo university ki parking ka faisla karta hai. Gate pe driver aata hai, system check karta hai ke isko park karne do ya nahi, kaunsi bay (parking space) do, aur agar mana kiya to exact wajah batata hai ke kaunse rule ki wajah se mana hua.

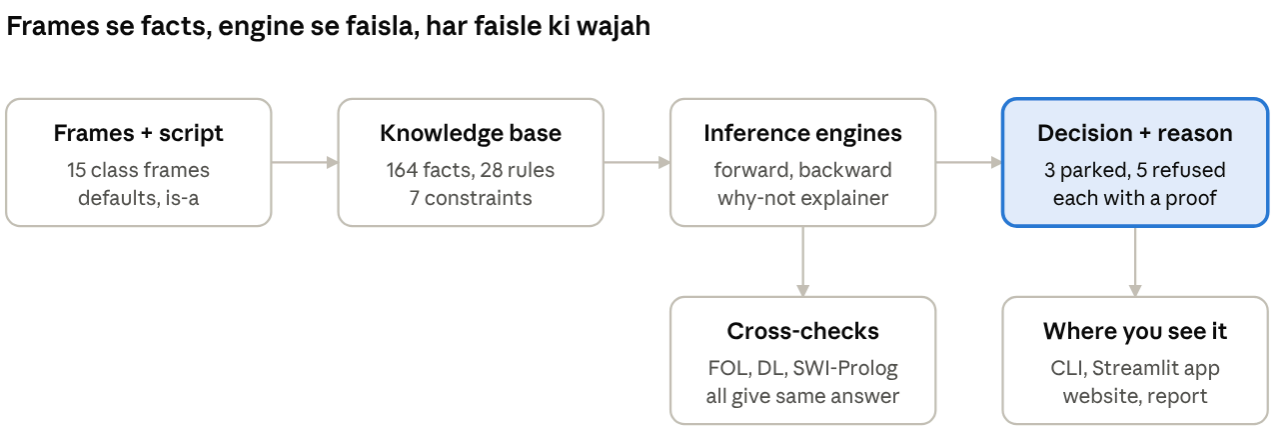
Yeh CT-351 Knowledge Representation and Reasoning (KRR) ka CCP (Complex Computing Problem) hai. Course ka main idea yeh hai: knowledge ko computer mein kaise likhein (representation) aur us knowledge se naye nateeje kaise nikalein (reasoning). Parking sirf ek example domain hai jis pe hum ne yeh sab apply kiya.

CCP guide ne humse yeh cheezein maangi thi, aur humne sab banayi:

- Frames aur scripts (concepts ko structure mein likhna)
- Semantic network (concepts ka connection wala diagram)
- First-order logic (FOL) aur description logic (DL) mein rules
- Horn clauses, aur un pe forward chaining aur backward chaining ke step-by-step traces
- Expressivity vs tractability ka analysis (zyada powerful logic vs tez chalne wali logic)
- Report, code snippets aur sample output

Humne iske upar extra bhi banaya: Python mein apna reasoning engine, command-line tool, Streamlit app, SWI-Prolog version, OWL ontology, 3D website, 70 automatic tests aur GitHub repo.

Ek line mein purpose: database sirf data store karta hai, lekin humara system rules se soch ke faisla karta hai aur har faisle ka "kyun" bhi bata sakta hai. Yahi KRR ka asal faida hai.



poora system ek nazar mein · frames se faisle tak

Baayen se daayen parho: frames facts banate hain, facts aur rules knowledge base mein jaate hain, engines faisla aur wajah nikalte hain, aur FOL, DL aur Prolog alag se check karte hain ke jawab sahi hai.

2. Scenario: subah ka rush (morning rush)

Humara main example yeh hai: Friday, 2 October 2026, subah 10 baje. 8 log gate pe parking maang rahe hain, aur 3 ko jagah milti hai, 5 ko mana hota hai.

Campus pe 3 zones hain:

- Student zone (zone_a), 07:00 se 22:00 tak khula. Bays: s_a1 standard, s_a2 standard (pehle se koi khara hai, occupied), s_a3 accessible (sirf badge walon ke liye), s_a4 motorbike, s_a5 standard (Zara ne reserve kiya hua).
- Employee zone (zone_e), 06:00 se 23:00. Bays: s_e1 standard, s_e2 EV charging, s_e3 standard (Ahmed ne reserve kiya hua).
- Visitor zone (zone_v), 08:00 se 17:00. Bay: s_v1 standard.

Ab har driver ki kahani:

Driver	Kaun hai	Kya maanga	Result	Kyun
Ahmed	Faculty, electric car	s_e3	Mil gayi	Aaj ki reservation uski hai
Fatima	Staff, motorbike	s_e1	Mana	Motorbike car wali standard bay mein fit nahi hoti
Zara	Student	s_a1	Mana	Sara ki priority zyada hai

Driver	Kaun hai	Kya maanga	Result	Kyun
Usman	Student	s_a1	Mana	Permit 15 Sep 2026 ko expire ho chuka
Ali	Student	s_a1	Mana	Sara ki priority zyada hai
Sara	Student, accessibility badge	s_a1	Mil gayi	Badge wali priority 3, sabse zyada
Hina	Visitor	s_v1	Mana	Uske paas visitor pass hi nahi
Bilal	Visitor, one-day pass	s_v1	Mil gayi	Pass sirf aaj (2 Oct) ke liye valid hai

Dhyan do: s_a1 ke liye 4 log lad rahe the (Zara, Usman, Ali, Sara). Usman to permit expire hone ki wajah se pehle hi bahar. Baqi teen mein Sara jeet gayi kyun ke uske paas accessibility badge hai. Yeh "conflict resolution" wala part hai jo aage section 11 mein detail se samjhaya hai.

Yeh 8 kahaniyan isliye chuni gayi ke har rule kahin na kahin kaam aaye: permit, zone, vehicle fit, badge, reservation aur priority, sab test ho jaate hain.

3. KRR ki basic cheezein

Poora project 4 cheezon pe khara hai: facts, rules, knowledge base aur inference engine.

Fact matlab ek seedhi si baat jo sach hai, jaise ek line ki khabar. Example:

```
student(ali).
owns(ali, car_ali).
permit_expiry(p_usman, 20260915).
```

Matlab: Ali student hai, Ali ki car car_ali hai, Usman ka permit 15 Sep 2026 ko khatam hota hai. Dates hum 20260915 jaisi numbers mein likhte hain taake computer aasani se compare kar sake (bara number = baad wali date).

Rule matlab "agar yeh sab sach hai, to yeh bhi sach hai". Example (R11):

```
authorized_driver(X) :- authorized(X, P).
```

Isko aise parho: "X authorized driver hai AGAR X kisi permit P se authorized hai." `:-` ka matlab "agar" hai, aur comma ka matlab "aur". Capital letters (X, P) variables hain, yani koi bhi value aa sakti hai.

Knowledge base (KB) matlab saare facts aur saare rules ek jagah. Humare main scenario mein 164 facts hain (142 frames se aaye, 12 policy table wale, 10 aaj ke context wale: date, hour aur 8 requests), 28 rules aur 7 integrity constraints.

Inference engine woh program hai jo facts pe rules chala ke naye facts nikalta hai. Jaise facts mein likha hai "Sara student hai" aur "Sara ke paas badge hai", engine rule laga ke nikalta hai "Sara ki priority 3 hai". Yeh naye facts derived facts kehlate hain. Main scenario mein engine 105 naye facts nikalta hai.

Code mein ek fact Python ka tuple hai, jaise `("owns", "ali", "car_ali")`. Facts sets mein rakhe hain, predicate ke naam se index karke, taake dhoondna tez ho.

Ek aur cheez yaad rakho: hum **closed world** maante hain. Matlab jo fact KB mein nahi likha, woh jhoota maana jaata hai. Hina ke liye `has_permit(hina, ...)` likha hi nahi, to system maan leta hai uske paas pass nahi hai.

4. Frames aur script (Huzaifa ka part)

Frame ko ek form ya template samjho. Jaise university ka admission form: upar likha hai "Student", aur neeche khaane hain (naam, roll number, department). Har khaana ek slot hai, aur jab Ali ka form bharte hain to woh ek instance ban jaata hai.

Humare project mein 15 class frames hain, jaise Person, Student, Faculty, Staff, Visitor, Vehicle, ParkingPermit, ParkingSpace, AccessibleSpace, MotorbikeSpace, ParkingZone aur Reservation. Code `krr/frames/` folder mein hai.

Frames ki zaroori cheezein:

- Slot: frame ka ek khaana. ParkingPermit ke slots: permitClass, status, startDate, expiryDate.
- Facet: slot ke baare mein extra info. Jaise slot ki type kya hai (number, symbol), kaunsi values allowed hain (range), aur kis predicate mein convert hoga.
- Default value: agar kuch nahi likha to yeh value maan lo. Jaise permit ka status default "active" hai, vehicle ka type default "car" hai, student zone default 07:00 pe khulta hai.
- Inheritance (is-a): child frame parent ki cheezein le leta hai. Student is-a Person, to Student ko Person ke saare slots mil jaate hain. AccessibleSpace is-a ParkingSpace, aur uski spaceType default "accessible" hai.
- Override: instance apni value de de to default nahi lagta. Jaise s_a2 ka status "occupied" likha hai, to default "vacant" nahi lagega. Fatima ki vehicle type "motorbike" likhi hai, default "car" override ho gaya.
- If-needed procedure: ek chhota function jo value tab calculate karta hai jab koi maange.

Slot ki value dhoondne ka order: pehle instance ki apni value, phir default (parent frames se), phir if-needed. `FrameSystem.get()` yeh kaam karta hai aur batata bhi hai value kahan se aayi.

Frames se facts kaise bante hain: har slot value ek fact ban jaati hai. Ali ke form se `student(ali)`, `owns(ali, car_ali)`, `has_permit(ali, p_ali)` jaise facts bante hain. Yeh `to_facts()` karta hai. Isi tarah 142 facts frames se aate hain.

Script kya hai: frame ek "cheez" ko describe karta hai, script ek "waqiye ki sequence" ko (Schank and Abelson ka idea). Jaise restaurant script: andar jao, order do, khao, bill do. Humari ParkingSession script ke 7 scenes hain:

1. Gate pe pohanchna
2. Permit check
3. Zone check
4. Bay check (vehicle fit, badge)
5. Availability (bay khaali hai ya reserved)
6. Allocation (conflict mein kaun jeeta)
7. Park karna

Script ke roles (driver, gate system, parking office), props (vehicle, permit, bay) aur entry conditions bhi defined hain. `python main.py script fatima s_e1` chalao to dikhata hai Fatima kaunse scene pe ruki (bay check pe, kyun ke motorbike fit nahi).

5. Semantic network (Maaz ka part)

Semantic network ek naqsha (graph) hai: boxes concepts hain aur arrows unke rishte. Jaise family tree hota hai, waise hi concepts ka tree aur jaal.

Teen main rishte:

- is-a: "yeh us ki ek qisam hai". Student is-a Person. Faculty is-a Employee. AccessibleSpace is-a ParkingSpace.
- has-a: "iske paas yeh hai". Person has-a Vehicle (owns), Person has-a ParkingPermit (has-permit).
- part-of: "yeh us ka hissa hai". ParkingSpace part-of ParkingZone (bay zone ka hissa), aur ParkingZone part-of ParkingFacility (zone_a, zone_e, zone_v sab ned_lot ka hissa hain).

Iske ilawa labelled arrows bhi hain: has-type (vehicle ya bay ki qisam), made-by (reservation kisne ki), for-space (reservation kis bay ki), grants (kaunsa permit kaunsa zone kholta hai) aur fits (kaunsi vehicle kis bay mein fit hoti hai).

Sabse achhi baat: humne network haath se nahi banaya. Code frames se khud generate karta hai: har class frame ek node, har is-a link ek is-a arrow, aur har slot jo kisi doosre frame ki taraf ishara karta hai, ek labelled arrow. Is ka faida: network ke labels hamesha frames aur rules ke naamon se match karte hain, galti ka chance nahi.

Inheritance network mein kaise chalti hai: is-a arrows ke saath upar jao. AccessibleSpace se ParkingSpace tak jao to woh saari properties mil jaati hain jo ParkingSpace ki hain.

Report mein ek worked example hai (section 3.2.3). Ali ki car ka rasta: car_ali has-type car, car fits standard, aur s_a5 has-type standard. Yeh chain wahi "vehicle fit" test hai jo rule R14 check karta hai. Matlab network pe arrow follow karna aur rule chalana ek hi soch hai, bas alag tarah likhi hai.

Network dekhne ke liye: `python main.py network` (concepts) ya `python main.py network ali` (Ali ke aas paas ka network). Website pe bhi 3D network hai jahan R13 aur R14 ke raste highlight hote hain.

6. First-order logic, FOL (Owais ka part)

FOL math wali bhasha hai jis mein rules bilkul saaf, bina confusion ke likhe jaate hain. English mein "har student ke paas permit hona chahiye" thoda ghol mol hai, FOL mein exact hota hai.

FOL ke hisse:

- Constants: specific cheezein, jaise ali, s_a1, zone_a.
- Variables: koi bhi cheez, jaise x, s, p.
- Predicates: koi property ya rishta, jaise student(x) "x student hai", owns(x, v) "x ke paas v hai".
- Connectives: ∧ (aur), ∨ (ya), ¬ (nahi), → (agar ... to).
- Quantifiers: ∀ (har ek ke liye, "for all") aur ∃ (kam se kam ek hai, "there exists").

Quantifiers sabse important hain. ∀ ka matlab "sab pe lagu", ∃ ka matlab "koi ek to hai".

Example, rule R06 FOL mein:

```
\forall p\,\forall b\,\forall e\,\forall d\;\big(permit\_status(p, active)
\land permit\_start(p, b) \land permit\_expiry(p, e) \land current\_date(d)
\land b \le d \land e \ge d\big) \rightarrow permit\_valid(p)
```

Aasan zabaan mein: har permit p ke liye, agar woh active hai aur aaj ki date uski start aur expiry ke beech mein hai, to permit valid hai.

Quantifier ka order bhi farq daalta hai. "Har bay ke liye koi zone hai jis mein woh hai" (∀s ∃z) sahi hai. Ulta likho "ek zone hai jis mein har bay hai" (∃z ∀s) to matlab badal gaya, aur yeh jhoot hai kyun ke humare 3 zones hain.

Humne 28 rules ke ilawa 12 aur FOL statements likhe (F01 se F12). Inn mein se kuch Horn clause mein likhe hi nahi ja sakte, isliye alag rakhe:

Code	Matlab	Kyun Horn mein nahi
F01	Har bay kisi zone mein hai	Conclusion mein ∃ hai
F02	Ek bay do zones mein nahi ho sakti	Equality (=) chahiye
F03	Har person student ya employee ya visitor hai	Conclusion mein ∨ hai
F04	Koi student aur employee dono nahi	Negation (¬) chahiye
F05	Ek bay do logon ko nahi milti	Equality chahiye
F10	Bina pass ka visitor authorized nahi	Classical negation
F12	Valid permit ka start aaj ya pehle, expiry aaj ya baad	R06 ka ulta half (only if)

Yeh 12 statements ek model checker har run pe check karta hai (`krr/logic/fol.py`). Main scenario pe sab sach hain. TC12 mein jaan boojh ke galat data hai (Ali student bhi aur faculty bhi), wahan F04 fail hota hai aur checker counterexample batata hai: x = ali. Yani checker galti pakadta hai.

`python main.py fol` chala ke sab statements aur unka result dekh sakte ho.

7. Description logic, DL (Maaz ka part)

DL bhi logic hai, lekin iska focus classes (concepts) pe hai: kaunsi class kis class ke andar aati hai, kaunsi classes kabhi mil nahi sakti. FOL se kam powerful hai, lekin isi wajah se computer isko hamesha jaldi aur pakka solve kar leta hai. Ontologies (jaise OWL, Protégé) isi pe chalti hain.

DL ke do hisse hain:

- TBox (terminology box): general rules, classes ke baare mein. Jaise "har Student ek Person hai". Humare TBox mein 45 axioms hain.
- ABox (assertion box): specific logon ke facts. Jaise "ali ek Student hai" (ali : Student). Inko membership assertions kehte hain.

Symbols ka matlab:

- ⊑ = "ke andar aata hai" (subsumption). Student ⊑ Person.

- ≡ = "bilkul barabar" (definition).
- ⊓ = aur, ⊔ = ya, ⊥ = khaali (aisa koi ho hi nahi sakta).
- ∃r.C = "kam se kam ek r rishta C se hai". ∀r.C = "jitne bhi r rishte hain, sab C se".

Axioms ki qismein, humare examples ke saath:

Qisam	Example	Matlab
Subsumption	T03: Faculty ⊑ Employee	Har faculty employee hai
Existential	T15: ParkingSpace ⊑ ∃locatedIn.ParkingZone	Har bay kisi na kisi zone mein hai
Universal	T19: AccessibleSpace ⊑ ∀allocatedTo.BadgeHolder	Accessible bay sirf badge walon ko
Definition	T25: AuthorizedStudent ≡ Student ⊓ ∃hasPermit.(ValidPermit ⊓ StudentPermit)	Authorized student = student jiske paas valid student permit
Disjointness	D01: Student ⊓ Employee ⊑ ⊥	Koi student aur employee dono nahi ho sakta

DL reasoner (`krr/logic/description_logic.py`) teen kaam karta hai:

1. Classification: TBox se naye subsumptions nikalna jo kisi ne likhe nahi. Jaise humne likha Faculty ⊑ Employee (T03) aur Employee ⊑ Person (T02). Reasoner khud nikalta hai Faculty ⊑ Person. Ek aur: AuthorizedStudent ⊑ AuthorizedDriver (T25 aur T28 se).
2. Consistency check: data mein koi contradiction to nahi. TC12 mein Ali student bhi aur faculty bhi likha hai, to D01 toot jaata hai aur reasoner "clash" report karta hai.
3. Realization: har banda kis class mein aata hai. Reasoner ke hisaab se AuthorizedDriver kaun hai, yeh bilkul wahi 8 logon ka result deta hai jo Horn rules ka authorized_driver deta hai.

Open world assumption: DL mein jo likha nahi woh "pata nahi" hai, jhoot nahi. Hina ke pass ka fact nahi hai, to DL kehta hai "maloom nahi". Humare Horn rules closed world maante hain aur kehte hain "pass nahi hai". Gate ko faisla karna hai, "pata nahi" se kaam nahi chalta, isliye final decision Horn rules ka hai aur DL classes aur consistency ke liye use hota hai.

Dekhne ke liye: `python main.py dl` .

8. Horn clauses aur 28 rules (Owais ka part)

Horn clause FOL ka ek simple roop hai: "agar yeh, yeh aur yeh sach hain, to sirf EK cheez sach hai". Conclusion mein ek hi atom hota hai, ∨ ya ∃ nahi. Technical definition: clause mein zyada se zyada ek positive literal.

Yeh simple kyun rakha? Kyun ke Horn clauses pe reasoning bohot tez aur pakki hoti hai (section 13 dekho). Isliye gate pe jo rules asal mein chalte hain, woh 28 Horn clauses hain, `krr/knowledge_base/rules.py` mein.

28 rules 8 groups mein bante hain:

Group	Rules	Kya karte hain
A. Class hierarchy	R01 se R05	Student/employee/visitor person hain, faculty/staff employee hain
B. Permits	R06 se R11	Permit valid hai? Role se match karta hai? Banda authorized hai?
C. Zones aur time	R12, R13	Zone abhi khula hai? Banda is zone mein ja sakta hai?
D. Suitability	R14 se R16	Vehicle bay mein fit hoti hai? Accessible bay ke liye badge hai?
E. Availability	R17 se R20	Bay khaali hai? Reservation isi bande ki hai?
F. Priority	R21 se R24	Badge = 3, faculty/staff = 2, student/visitor = 1
G. Conflicts	R25 se R27	Kaun haara (zyada priority wala jeeta, barabar ho to pehle aane wala), aur kisi ki do reservations to nahi (double booking)
H. Decision	R28	Final: bay de do agar sab theek aur koi conflict nahi haara

Final rule R28 aisa dikhta hai:

```
allocate(X, S) :- requests(X, S, T), can_allocate(X, S),
                  not loses(X, S), not double_booked(X).
```

Matlab: X ko bay S do agar X ne S maangi, S X ko di ja sakti hai, X kisi conflict mein haara nahi, aur X ki do reservations nahi.

English requirement ko Horn rule mein badalne ke 4 steps (report section 5.4):

1. If-then sentence banao. "Expired permit se parking nahi" ko positive bolo: "permit valid hai agar active hai aur dates ke andar hai".
2. FOL formula likho (section 6 wala R06).
3. Check karo conclusion ek hi atom hai (permit_valid(p)). Haan, to Horn hai.
4. Prolog syntax mein likh do.

Negative requirements ko hum positive bana ke likhte hain. "Occupied bay allocate nahi hogi" ki jagah "sirf vacant bay, ya apni reserved bay, allocate hogi" (R19, R20). Aur "jab tak koi zyada priority wala na ho" (unless) wali baat ke liye negation as failure use hua (section 11).

Ek aur design trick: "student student zone mein, employee employee zone mein" har combination ke liye alag rule likhne ki jagah humne ek policy table banayi: `grants(student_permit, student_zone)` jaise facts. Ab ek hi rule R13 sab handle karta hai, aur naya zone add karna ho to sirf ek fact add karo, rule nahi.

9. Forward chaining (Owais ka part)

Forward chaining "data se shuru" hoti hai (data-driven). Jo facts pata hain unse shuru karo, rules lagao, naye facts nikalo, phir un naye facts pe dobara rules lagao. Tab tak karte raho jab tak kuch naya na nikle.

Misaal: ghar mein pehle dekha "zameen geeli hai" aur "aasman pe badal hain", in se nikala "baarish hui", phir us se nikala "chhatri chahiye thi". Facts se aage ki taraf chalte gaye.

Algorithm, aasan steps mein (`krr/reasoning/forward.py`):

1. Saare facts working memory mein daal do (main scenario: 164 facts).
2. Ek cycle mein har rule ko check karo: jitne bhi tareeqon se uski body facts se match hoti hai, woh sab conflict set mein jaate hain.
3. Jo matches koi NAYA fact dete hain, woh fire karo aur naye facts add kar do.
4. Agla cycle. Jab kisi cycle mein kuch naya na aaye, woh fixpoint hai, ruk jao.

Kuch zaroori terms:

- Conflict set: is cycle mein jitne rule instances match hue, sab ki list.
- Refraction: jo fact pehle se pata hai usko dobara fire nahi karte. Isi wajah se loop nahi banta.
- Fixpoint: woh haalat jahan koi rule kuch naya nahi de raha.
- Snapshot: har cycle cycle ke shuru wale facts pe hi match karti hai. Isliye trace saaf rehta hai: cycle 2 mein sirf cycle 1 tak ke facts use hote hain.

Strata (layers): R28 mein "not loses" hai. "Not" tab hi sahi kaam karta hai jab loses ke saare facts pehle hi nikal chuke hon. Isliye engine do layers mein chalta hai: stratum 0 mein R01 se R27 (sab kuch jo "not" ke bina hai), aur jab woh poora ho jaaye tab stratum 1 mein R28. Isko stratified negation kehte hain.

Main scenario ka asli trace:

Cycle	Stratum	Naye facts	Kaunse rules fire hue
1	0	35	R01, R03-R07, R09, R12, R17, R18, R23, R24
2	0	11	R02, R08, R10, R21, R22
3	0	10	R10, R11, R13
4	0	17	R11, R13, R14
5	0	14	R14, R15, R16
6	0	9	R15, R19, R20
7	0	6	R19, R20, R25
8	0	0	stratum 0 ka fixpoint
9	1	3	R28: Ahmed, Sara, Bilal ko bay
10	1	0	fixpoint, khatam

Kul 105 naye facts, 10 cycles. Dekho kaise pehle basic cheezein nikalti hain (person, permit valid, zone open), phir authorized, phir zone access, phir suitable bay, phir conflicts, aur aakhir mein allocate. Yeh layer by layer upar jaana hi forward chaining hai.

Termination kyun pakki hai: humare paas function symbols nahi hain, to constants limited hain (ali, s_a1 waghera). Limited constants se limited facts ban sakte hain, aur refraction har fact ek hi baar add karta hai. Isliye engine hamesha ruk jaata hai.

Kab use karo: jab saare faislay ek saath chahiye (poore subah ke rush ka result), forward chaining best hai.

Dekhne ke liye: `python main.py forward`. Website ke "How it decides" section mein bhi har cycle step by step dikhti hai.

10. Backward chaining (Zain ka part)

Backward chaining "sawal se shuru" hoti hai (goal-driven). Pehle sawal pucho, jaise "kya Sara ko s_a1 milegi?", phir peeche ki taraf jao: is ko sach karne ke liye kya chahiye, aur us ke liye kya chahiye, jab tak stored facts tak na pohanch jao.

Misaal: detective pehle shak karta hai "qatil butler hai", phir saboot dhoondta hai jo is baat ko sabit karein. Yeh wahi tareeqa hai jo Prolog use karta hai (SLD resolution).

Algorithm, aasan steps mein (`krr/reasoning/backward.py`):

1. Goal lo, jaise allocate(sara, s_a1).
2. Koi stored fact match karta hai? Haan to goal sabit.
3. Nahi to woh rule dhoondo jiska head goal se match kare (R28). Uski body ke saare hisse naye subgoals ban jaate hain.
4. Har subgoal ko isi tarah sabit karo, depth first, left se right.
5. Koi raasta fail ho jaaye to wapas aao aur agla option try karo. Isko backtracking kehte hain.

Zaroori terms:

- Unification: do cheezon ko barabar banane ke liye variables ki values dhoondna. Goal can_allocate(ahmed, S) aur fact ka match hua to S = s_e3 nikla. Matlab "S kya ho ke dono same ho jaayein".
- Substitution: variables ki values ki list, jaise {X = sara, S = s_a1}.
- Backtracking: ek raasta band ho to pichhle choice pe wapas jaa ke doosra try karna. Code mein yeh Python generator se hota hai: "agla answer do" bolo to woh doosra raasta try karta hai.
- Renaming (standardise apart): har baar rule use karne se pehle uske variables ka naam badalte hain, jaise P se P_5. Warna do alag rule uses ke variables aapas mein mix ho jaate. Trace mein jo P_5, T_1 dikhte hain, woh isi wajah se hain.
- Loop check: agar koi goal apne hi andar dobara aa jaaye to us raaste ko rok do. Saath mein 40 depth ki limit bhi hai, double safety.
- Proof tree: jo raasta kaamyab hua, woh ek tree ki shakal mein. Upar goal, neeche rules, aakhir mein facts.

Asli examples:

- `python main.py prove "can_allocate(ahmed, S)"` teen answers deta hai: S = s_e1, s_e2, s_e3. Yani Ahmed teeno employee bays ke liye eligible hai, lekin usne maangi s_e3 thi.
- allocate(sara, s_a1) sabit karne mein 2,240 steps lagte hain. Itne zyada kyun? Kyun ke "not loses(sara, s_a1)" check karne ke liye engine har competitor (Zara, Usman, Ali) ke liye can_allocate dobara sabit karta hai. Tabling naam ki technique se yeh repeat kaam bach sakta hai (future work).

Forward vs backward, kab kaunsa:

	Forward chaining	Backward chaining
Shuru kahan se	Facts se	Sawal (goal) se
Kya nikalta hai	Sab kuch ek saath	Sirf woh jo sawal ko chahiye
Best kab	Poore subah ke saare faislay	Ek driver gate pe khara hai, usi ka faisla aur proof
Kamzori	Kuch kaam bekaar (jaise Hina ki priority, jo kisi kaam nahi aayi)	Kuch kaam repeat hota hai

Dono engines alag alag likhe gaye hain, aur tests check karte hain ke dono har scenario mein bilkul same facts nikalein. Yeh humara sabse strong correctness proof hai (section 15).

11. Priority, conflicts, negation as failure aur why-not

Jab ek bay ke liye kai log ho, to faisla priority se hota hai, aur barabar priority ho to jo pehle aaya woh jeeta.

Priority rules (R21 se R24):

- Accessibility badge wala: 3 (sabse zyada)
- Faculty ya staff, badge ke bina: 2
- Student ya visitor, badge ke bina: 1

Conflict rules:

- R25: Y haar jaata hai agar koi doosra X bhi wahi bay maange, X ko woh bay di ja sakti ho, aur X ki priority zyada ho.
- R26: priority barabar ho to jiska request time pehle hai woh jeeta (first come, first served).

Main scenario mein s_a1 ki ladai: Sara (badge, priority 3) ne Zara aur Ali (dono priority 1) ko R25 se haraya. Usman to ladai mein shamil hi nahi, kyun ke uska permit expired hai, to woh can_allocate hi nahi. TC7 test case mein Zara (08:50) aur Ali (09:05) dono priority 1 hain, to R26 se Ali Zara se bhi haarta hai, kyun ke Zara pehle aayi.

Negation as failure (NAF): R28 mein "not loses(X, S)" likha hai. Iska matlab: agar engine loses(X, S) sabit NAHI kar sakta, to maan lo X haara nahi. "Sabit nahi ho saka" ko "jhoot" maan lena, yeh closed world wali soch hai. Ek masla hai: agar "not" pehle check ho jaaye aur loses wale facts baad mein nikalein, to galat faisla ho sakta hai. Isi liye stratification

(section 9): pehle saare loses facts poore nikalo (stratum 0), phir R28 chalao (stratum 1). Pura project mein "not" sirf yahin use hua hai, aur code aisa koi rule set accept hi nahi karta jis mein recursion "not" ke through ho.

Why-not explainer (`krr/reasoning/explain.py`): jab koi goal fail ho, yeh peeche jaa ke dhoondta hai ke kaunsi pehli condition poori nahi hui. Har level pe batata hai kaunsa rule kis jagah ruka. Fatima ka asli output:

```
allocate(fatima, s_e1) cannot be proved. Reasons, rule by rule:
  R28 blocked at can_allocate(fatima, s_e1)
    R19 blocked at eligible_for_space(fatima, s_e1)
      R15 blocked at suitable_space(fatima, s_e1)
        R14 blocked at fits(motorbike, standard)  (no such fact is stored)
```

Neeche se upar parho: fits(motorbike, standard) ka fact hi nahi hai, isliye R14 ruka, isliye bay suitable nahi, isliye eligible nahi, isliye allocate nahi. Sabse neeche wali line "root cause" hai. Usman ke liye root cause aata hai "20260915 >= 20261002 false", yani expiry date aaj se pehle thi.

Yahi explainer website pe har refused driver ki wajah likhta hai, aur `python main.py why "allocate(usman, s_a1)"` se command line pe bhi dekh sakte ho.

12. Integrity constraints aur consistency

Integrity constraint ek "aisa kabhi nahi hona chahiye" wala rule hai. Rules naye facts banate hain, constraints sirf check karte hain ke koi galat haalat to nahi ban gayi. Agar constraint ki body sabit ho jaaye, to woh toot gaya (violation).

Humare 7 constraints:

Code	Kya kabhi nahi hona chahiye
IC1	Ek bay do logon ko mil jaaye
IC2	Ek bande ko do bays mil jaayein
IC3	Occupied bay kisi ko allocate ho jaaye
IC4	Koi student bhi ho aur employee bhi
IC5	Koi student bhi ho aur visitor bhi
IC6	Koi employee bhi ho aur visitor bhi
IC7	Koi faculty bhi ho aur staff bhi

Prolog mein inko "denial clause" ki tarah likha hai, jaise `violation(ic3) :- allocate(X, S), space_status(S, occupied).` Phir `consistent` ka matlab hai "koi violation sabit nahi hota".

Forward chaining ke baad engine saare constraints check karta hai. Main scenario aur 11 test cases mein koi violation nahi. TC12 mein jaan boojh ke galat data daala (Ali student bhi aur faculty bhi), wahan IC4 toot jaata hai. Test yahi check karta hai ke system yeh galti pakde.

Galat data teen alag tareeqon se pakda jaata hai, aur teeno same baat kehte hain:

1. Horn engine: IC4 violation.
2. FOL model checker: F04 fail, counterexample x = ali.
3. DL reasoner: D01 (Student ⊓ Employee ⊑ ⊥) ka clash.

Yeh dikhata hai ke humari paanchon representations ek doosre ke saath consistent hain. Viva mein agar pucha jaaye "paradigms ko consistent kaise rakha", to yahi jawab hai: ek hi vocabulary, frames se generated facts, aur har run pe cross-checks.

13. Expressivity vs tractability (Maaz ka part)

Yeh CCP ka sabse "soch" wala topic hai. Seedha rule: jitni zyada powerful logic (zyada baatein keh sake), utna mushkil usko computer pe solve karna. Humne har kaam ke liye utni hi power li jitni zaroori thi.

Do lafz:

- Expressivity: logic kitni qisam ki baatein likh sakti hai.
- Tractability: computer kitni jaldi aur pakka jawab de sakta hai.

Chaaron logics ka muqabla:

Logic	Kya keh sakti hai	Masla
Propositional	Sirf seedhe sach/jhoot statements	Variables nahi. "Valid permit wala student student zone mein ja sakta hai" har student, har permit, har zone ke liye alag likhna padega. 8 log hon to theek, 8,000 hon to laakhon sentences
FOL	∀, ∃, =, ∨, ¬, sab kuch	Itni powerful ke general mein entailment undecidable hai: computer kabhi kabhi hamesha chalta rahe aur jawab na de

Logic	Kya keh sakti hai	Masla
Description logic	Classes, ∃ aur ∀ restrictions, disjointness	Decidable (jawab hamesha milega), lekin rules jaise "X ki priority Y se zyada" aasani se nahi keh sakti
Horn clauses	Agar-to rules, ek conclusion	Conclusion mein ∃ ya ∨ nahi aa sakta, lekin reasoning tez aur hamesha khatam hoti hai

Humne kya choose kiya aur kyun:

- Gate ka faisla Horn clauses se, kyun ke gate pe jawab tez, pakka aur explanation ke saath chahiye. Horn pe forward chaining polynomial time mein chalti hai aur hamesha ruk jaati hai (section 9).
- Jo Horn mein nahi likha ja sakta (F01 "har bay kisi zone mein hai" mein ∃, F03 mein ∨), woh FOL statements ki tarah rakha aur model checker se check kiya. Model checking ek finite model pe hoti hai, isliye woh bhi khatam hoti hai.
- Classes aur unke rishte DL mein, kyun ke DL decidable hai aur classification (naye subsumptions dhoondna) acha karti hai.
- Defaults aur structure frames mein, kyun ke logic mein "default jab tak override na ho" likhna mushkil hai.

Isko multi-paradigm KRR kehte hain: ek hi problem, har hissa us representation mein jo uske liye best hai, aur sab ek hi vocabulary share karte hain. Report ka section 8.2.2 yahi analysis detail mein karta hai.

14. Code: kaunsi file kya karti hai, aur kaise bana

Sab kuch Python mein hai, aur reasoning engine sirf Python ki standard library use karta hai. Koi library install kiye bina chal jaata hai, aur har line humari likhi hai, isliye viva mein har cheez explain kar sakte hain.

Sabse zaroori design idea: rules sirf EK jagah likhe hain (`krr/knowledge_base/rules.py`). Prolog file, OWL ontology, traces, report ki tables aur website ka data, sab code se generate hota hai. Isliye kuch bhi out of sync nahi ho sakta.

Folders aur unka kaam:

Folder ya file	Kya hai
krr/core/	Basic engine parts: terms, parser (text ko rule mein badalna), unification, knowledge base

Folder ya file	Kya hai
krr/knowledge_base/	Facts, 28 rules, 7 constraints, aur scenarios (main + 12 test cases)
krr/frames/	Frame system, saari frames aur instances (ontology.py), ParkingSession script
krr/semantic_net/	Frames se network banana
krr/logic/	FOL statements + model checker, DL TBox + reasoner
krr/reasoning/	forward.py, backward.py, explain.py (why-not)
krr/export/	Prolog, OWL aur Mermaid diagram generators
krr/site_export.py	Website ke liye data banana, aur "add a driver" ke facts
main.py	Command-line tool
app.py, app/	Streamlit web app
tests/	70 automatic tests
tools/	Build scripts: report, figures, website data
website/	Website (HTML, CSS, JavaScript, 3D)
docs/	Report, figures, traces, viva notes

Code ke andar cheezein kaise represent hain:

- Atom (fact) = tuple, jaise ("owns", "ali", "car_ali"). Tuples hash ho sakte hain, isliye sets mein tez dhoondhe jaate hain.
- Variable = string jo capital letter se shuru ho, jaise "X".
- Substitution = dictionary, jaise {"X": "ali"}.
- Rule = Prolog jaisa text likha, parser ne Rule object bana diya, aur load hote waqt "safety" check hota hai (head ka har variable body mein bhi ho).
- Frames = Python dataclasses (Slot, Frame, FrameSystem).

Banane ka order kuch aisa tha: pehle core (parser, unification), phir domain (frames, facts, rules), phir dono engines, phir CLI aur app, phir tests, phir Prolog/OWL export, report aur aakhir mein website. GitHub pe commits bhi isi order mein hain.

15. Testing: kaise pata ke sab sahi hai (Zain ka part)

Testing do level pe hai: 12 test cases (scenarios) jo har requirement check karte hain, aur 70 unit tests jo code ke har hisse ko check karte hain. Sab pass hain.

12 test cases (har ek ek chhota scenario hai jiska expected result pehle se likha hai):

Case	Situation	Kahan rukta hai
TC1	Valid student, valid permit, khaali student bay	Allocate ho jaata hai
TC2	Expired permit (Usman)	R06
TC3	Visitor bina pass (Hina)	R09
TC4	Bay pehle se occupied	R19/R20
TC5	Student employee bay maange	R13 (zone access)
TC6	Eligible driver, lekin suitable bay nahi (Fatima ki motorbike)	R14
TC7	Teen log ek bay maangein	R25, R26
TC8	Accessible bay bina badge	R16
TC9	Conflicting reservations (double booking)	R27
TC10	Visitor zone 19:00 pe band	R12
TC11	Kisi aur ki reserved bay maangna	R20
TC12	Galat data: student bhi aur faculty bhi	IC4

Har test case mein kuch goals ka expected answer likha hai (kul 49 goal checks). Har goal forward chaining se bhi aur backward chaining se bhi check hota hai, aur dono ka jawab expected se match hona chahiye. `python main.py cases` yeh sab chala ke dikhata hai.

70 unit tests (6 files, `python -m unittest discover -s tests`):

File	Tests	Kya check karta hai
test_core.py	13	Parser, unification, knowledge base
test_representation.py	22	Frames, defaults, script, network, FOL, DL
test_reasoning.py	16	Forward, backward, explainer, saare scenarios
test_consistency.py	10	Sab representations ek doosre se match, Prolog round trip

File	Tests	Kya check karta hai
test_site_export.py	7	Website ka clock aur "add a driver"
test_prolog.py	2	SWI-Prolog wahi jawab de jo Python engine

Sabse strong tests yeh hain:

- Dono engines alag likhe gaye, phir bhi har scenario mein har derived predicate ke liye bilkul same facts dete hain.
- Forward chaining ka final result har rule ka "model" hai: koi rule ab kuch naya nahi de sakta.
- Har firing sirf un facts ko use karti hai jo pehle se pata the.
- SWI-Prolog, jo bilkul alag system hai, wahi conclusions nikalta hai (section 16).

GitHub Actions har push pe yeh saare tests Python 3.10 aur 3.12 pe khud chalata hai. Humne speed ya real traffic pe accuracy measure nahi ki, kyun ke real data nahi tha. Humara claim logical correctness ka hai, humare scenarios pe.

16. Prolog aur OWL

Humara knowledge base do aur famous formats mein bhi export hota hai, taake doosre tools bhi isko chala sakein. Dono files code khud generate karta hai, haath se nahi likhi.

Prolog (`prolog/smart_parking.pl`): Prolog ek programming language hai jo khud backward chaining karti hai. Humare 28 rules Prolog syntax mein hi likhe hain, isliye export seedha hai: saare 164 facts, 28 rules, 7 constraints (violation ki shakal mein), aur do helpers `consistent` aur `show_allocations`. Python ka `not` Prolog mein `\+` ban jaata hai.

SWI-Prolog mein chalana:

```
swipl prolog/smart_parking.pl
?- allocate(sara, S).
S = s_a1.
?- show_allocations.
allocate(ahmed, s_e3)
allocate(bilal, s_v1)
allocate(sara, s_a1)
?- consistent.
true.
```

Singleton variables: jo variable rule mein sirf ek baar aaye, Prolog us pe warning deta hai. Humare generator ne aise variables ke aage underscore laga diya (jaise `authorized(X, _P)`). Matlab same hai, bas warning nahi aati.

Teesra independent check: `python main.py prolog` har scenario (main + 12) ke liye Prolog file banata hai, SWI-Prolog mein chalata hai, aur Prolog jo bhi sabit kar sakta hai woh Python engine ke result se compare karta hai. Result: 13 scenarios, 0 differences. Prolog bilkul alag system hai jo humne nahi likha, phir bhi same jawab deta hai. Yeh strong saboot hai ke humara engine sahi hai. GitHub Actions bhi har push pe SWI-Prolog install karke yeh check chalata hai.

OWL (`ontology/smart_parking.ttl`): OWL web ka standard ontology format hai (Turtle syntax). Is mein classes, unke subclass rishte aur individuals hain. Isko Protégé software mein khol ke DL reasoner chala sakte ho, aur class hierarchy wahi dikhegi jo humare DL reasoner ne nikali.

17. Command-line tool aur Streamlit app

System ko chalane ke do tareeqe hain: terminal mein commands (CLI), ya browser mein Streamlit app.

CLI commands (`main.py`, project folder mein terminal khol ke):

Command	Kya dikhata hai
python main.py	Menu, number daba ke cheezein chalao
python main.py demo	Poore subah ke rush ka walkthrough
python main.py forward	Forward chaining, cycle by cycle
python main.py prove "allocate(sara, s_a1)"	Backward chaining ka poora trace aur proof tree
python main.py why "allocate(usman, s_a1)"	Goal kyun fail hua
python main.py cases	12 test cases, dono engines
python main.py prolog	SWI-Prolog vs Python check
python main.py kb / frames / network / fol / dl	Facts aur rules / frames / network / FOL / DL
python main.py script fatima s_e1	Script scene by scene
python main.py scenarios	Saare scenarios ki list

Koi bhi command doosre scenario pe chalani ho to aakhir mein `-s` lagao, jaise `python main.py forward -s tc07_conflict`.

Streamlit app: `pip install -r requirements.txt` ek baar, phir `streamlit run app.py`. Browser mein 10 tabs khulte hain:

1. Overview: subah ke faislay
2. Knowledge base: saare facts aur rules
3. Frames and script
4. Semantic network
5. FOL
6. DL
7. Forward chaining: slider se cycle 1 se 10 tak chalao
8. Backward chaining: apna sawal likho, trace, proof tree aur why-not dekho
9. Test cases
10. Playground: date, hour aur requests badal ke dobara reasoning chalao

Viva demo ke liye CLI fast aur bharosemand hai (internet nahi chahiye). Streamlit tab achha hai jab examiner ko dikhana ho ke cheezein visual mein kaise lagti hain.

18. Website

Website yahan hai: https://smart-parking-krr.vercel.app. Yeh project ko dikhane ka sabse attractive tareeqa hai, aur iska har number aur har faisla asli engine se aata hai.

Website kaise bani:

- Plain HTML, CSS aur JavaScript. 3D ke liye Three.js library.
- `tools/build_site.py` engine chalata hai aur saare faislay, proofs, cycles aur test results `website/data.js` mein likh deta hai. Website wahi dikhati hai, apni taraf se kuch nahi banati.
- Vercel pe hosted hai (free hosting). Har update pe "deploy" kiya.

Website ke hisse (upar se neeche):

1. 3D car park: "Replay the morning" dabao to cars aati hain, gate pe check hoti hain, allowed cars reverse kar ke bay mein lagti hain, refused cars mud ke chali jaati hain. Har car ke upar wajah ka bubble aata hai.
2. Key numbers: facts, rules, cycles waghera.
3. How it decides: forward chaining ki har cycle step by step. 3D mein "knowledge core" se facts udte dikhte hain.
4. Ask why: koi bhi driver chuno, allowed hai to proof tree, refused hai to pehli failed condition. 3D mein proof tower banta hai.
5. Network: 3D semantic network, R13 aur R14 ke raste highlight.
6. Representations: paanchon representations ki figures.
7. Tests: 12 test cases, har ek ka beacon green hota hai.
8. Team.

Do "live" features jo asli Python engine browser mein chalate hain:

- Scenario clock: koi bhi date aur hour chuno, engine dobara faisla karta hai. Raat 23:00 pe saare zones band, sab refused. 3 Oct 2026 ko Bilal ka one-day pass expire. 1 Jul 2027 ko saare student permits expire.
- Add a driver: form bharo (naam, role, vehicle, permit, bay, time, badge). Form ke answers facts ban jaate hain, jaise student(omar) aur requests(omar, s_a1, 905). Engine sab ka dobara faisla karta hai aur Ask why mein naye driver ka proof dikhata hai. Viva mein examiner koi bhi case bole, aap 10 second mein live jawab dikha sakte ho.

Yeh browser mein Python kaise chalta hai: Pyodide naam ki cheez hai jo Python ko WebAssembly mein browser ke andar chala deti hai. Pehli baar thoda time lagta hai (lagbhag 10 second, internet chahiye), phir har jawab ek second se kam mein. Koi server nahi, sab aapke browser mein hota hai.

Themes aur settings:

- Moon button (dark, default): neon raat wala look.
- Sun button (light): din ki roshni jo clock ke hisaab se badalti hai (subah, shaam, raat).
- 3D button: 3D band kar ke simple 2D page.
- Quality button: Auto, High, Low. Kamzor laptop pe Auto khud Low kar deta hai.

Cars ke models Kenney ki free (CC0) Car Kit se hain, footer mein credit diya hai.

19. Report: kya hai aur kaise bani

Report 33 pages ki hai, Word (.docx) aur PDF dono mein, `docs/report/` folder mein. Website se bhi download hoti hai.

Report ka structure (department ke CT-351 CCP report template ke mutabiq, wahi 11 sections):

- Cover page, grading rubric sheet (4 members ke naam aur roll numbers) aur report contents
- 1. Introduction: selected domain, problem statement, objectives
- 2. Domain analysis: requirements, entities, relationships, challenges, traditional database kyun kaafi nahi (2.2.4)
- 3. Knowledge representation: frames aur script (3.1), semantic network (3.2)
- 4. Formal logic representation: FOL (4.1), description logic (4.2)
- 5. Horn clause knowledge base: poore facts, rules, denial clauses, conversion (5.4), Prolog form (5.5)
- 6. Inference and reasoning: forward chaining (6.1), backward chaining (6.2), traces ke saath
- 7. Implementation: tools, code, execution screenshots, website (7.2.9)
- 8. Results and discussion: results (8.1), expressivity vs tractability (8.2.2), limitations (8.2.5)
- 9. Conclusion aur future improvements
- 10. References
- 11. Team participation: har member ka hissa, declaration aur Group Representative ka signature

Report kaise generate hoti hai: hum ne report Word mein haath se type nahi ki. Text `docs/report/report.md` mein likha hai, aur us mein special tags hain jaise `{{stat facts}}` ya `{{horn_table}}`. `tools/build_report.py` code chalata hai, numbers aur

tables asli engine se bharta hai, aur university ke template pe Word file banata hai. Phir `tools/finalize_report.py` Word khol ke contents aur page numbers bharta hai aur PDF banata hai.

Figures `tools/figures.py` se bante hain, print size pe aur 300 dpi, taake text clear padha ja sake.

Faida: agar koi rule badla to `python tools/build_all.py` chalao, aur report, traces, Prolog, OWL aur website ka data sab khud update ho jaata hai. Report aur code kabhi alag baat nahi karte. Jaise jab Bilal ke pass wala gap fix kiya, to report ke numbers (164 facts, 70 tests) khud update ho gaye.

20. GitHub aur CI

Poora project public GitHub repo mein hai: https://github.com/ZainDev04/smart-parking-krr. Contributor sirf ZainDev04 hai.

Repo mein kya hai: saara code, report (Word, PDF aur source), Prolog aur OWL files, website, tests, viva notes, aur README. README project ka front page hai: kya hai, kaise chalana hai, ek asli example (Fatima ka why-not), screenshot aur live website ka link.

Git kya hai, ek line mein: git har change ka record rakhta hai (commit). Humare commits usi order mein hain jis mein project bana: core engine, domain model, reasoning engines, interfaces, tests, generated files, report, website, README, aur aakhir mein Prolog check.

CI (Continuous Integration) kya hai: har baar jab code GitHub pe push hota hai, GitHub Actions khud ek fresh Linux machine pe:

1. Python 3.10 aur 3.12 install karta hai
2. SWI-Prolog install karta hai
3. Saare 70 unit tests chalata hai
4. `python main.py cases` chalata hai (12 test cases)
5. `python main.py prolog` chalata hai (Prolog vs Python)

Sab pass ho to repo pe green "tests passing" badge dikhta hai. Iska matlab: koi bhi kabhi bhi dekh sakta hai ke project abhi bhi sahi chal raha hai, sirf humare computer pe nahi.

Kisi bhi computer pe project chalane ke liye:

```
git clone https://github.com/ZainDev04/smart-parking-krr.git
cd smart-parking-krr
python main.py demo
```

21. Viva ki tayyari

Rubric ka criterion 5 har student ko alag marks deta hai, isliye har member apna part bina notes ke samjha sake, yeh zaroori hai. Detail wali guide `docs/viva/viva_guide.md` mein hai, yahan short version hai.

Kaun kya present karega:

Member	Part	Demo
Syed Mohmmad Huzaifa	Problem, knowledge acquisition, frames, script	Frames tab mein s_a3 kholo, "default from AccessibleSpace" dikhao; `python main.py script fatima s_e1`
Maaz ur Rehman	Semantic network, DL, expressivity vs tractability	`python main.py dl`: Faculty ⊑ Person inferred; TC12 ka clash
Owais Tariq	FOL, Horn clauses, forward chaining	`python main.py forward`: 10 cycles; R06 ka FOL se Horn conversion
Shaikh Muhammad Zain	Backward chaining, testing, interfaces, website	`python main.py prove "can_allocate(ahmed, S)"`, `python main.py cases`, `python main.py prolog`, website pe "Add a driver"

Viva se pehle checklist:

- [ ] Presentation laptop pe Python aur SWI-Prolog install, `python main.py cases` aur `python main.py prolog` chala ke check
- [ ] Website laptop aur projector pe khol ke dekho; dark scene muddy lage to sun button
- [ ] Viva se pehle website pe clock ek baar hilao taake Python engine load ho jaaye
- [ ] Har member apna section is guide aur viva_guide.md se 2 baar bol ke practice kare

Aam sawal, chhote jawab:

- Project kya karta hai? Campus parking ka faisla rules se karta hai aur har refusal ki exact wajah batata hai.
- Database kyun nahi? Database data rakhta hai, reasoning nahi karta. Hum rules se naye facts nikalte hain aur "kyun" bata sakte hain.
- Knowledge base kya hai? Facts aur rules ek jagah. Humare paas 164 facts, 28 rules, 7 constraints.
- Frame kya hai? Ek template jiske slots hain, defaults hain, aur inheritance hai.

- is-a aur part-of mein farq? is-a = qisam (Student is-a Person), part-of = hissa (zone part-of facility).
- Horn clause kya hai? Agar-to rule jiska conclusion ek hi atom ho. Tez aur hamesha khatam hone wali reasoning.
- Forward vs backward? Forward facts se sab nikalti hai, backward ek sawal se peeche jaati hai. Saare faislay = forward, ek driver ka proof = backward.
- Unification kya hai? Variables ki aisi values dhoondna ke do expressions same ho jaayein.
- Negation kaise handle ki? Negation as failure, stratification ke saath: pehle saare loses facts, phir R28.
- FOL sab ke liye kyun nahi? General FOL undecidable hai. Gate ko pakka aur tez jawab chahiye, isliye Horn.
- Open world vs closed world? DL open world (jo nahi likha woh pata nahi), Horn closed world (jo nahi likha woh jhoot).
- Engines sahi kaise pata? Dono engines alag likhe, har scenario mein same result. Plus SWI-Prolog bhi same result deta hai. Plus 49 goal checks aur 70 unit tests.
- Agar naya zone add karna ho? Sirf ek grants fact aur zone ke facts, koi rule nahi badalta.

22. Mushkil words ka asaan matlab

Word	Asaan matlab
ABox	DL ka woh hissa jis mein specific logon ke facts hain (ali : Student)
Atom	Ek chhota fact ya condition, jaise student(ali)
Axiom	Ek basic rule jo sach maana jaata hai (DL mein)
Backtracking	Ek raasta fail ho to wapas aa ke doosra try karna
Backward chaining	Sawal se shuru kar ke peeche facts tak jaana
Closed world	Jo likha nahi, woh jhoot maana jaata hai
Conflict set	Ek cycle mein jitne rules match hue, unki list
Constant	Ek specific cheez ka naam, jaise ali ya s_a1
Decidable	Computer hamesha jawab de dega, kabhi atkega nahi
Default	Jab kuch na likha ho to jo value maan li jaaye

Word	Asaan matlab
Derived fact	Engine ka rules se nikala hua naya fact
Description logic	Classes aur unke rishton ki logic, FOL se kam powerful lekin hamesha solve hone wali
Disjoint	Do classes jinka koi common member nahi ho sakta
Expressivity	Logic kitni qisam ki baatein keh sakti hai
Facet	Slot ke baare mein extra info (type, range, default)
Fixpoint	Woh haalat jab koi rule kuch naya nahi de raha
FOL	First-order logic: variables, ∀, ∃ wali logic
Forward chaining	Facts se shuru kar ke aage naye facts nikalna
Frame	Ek template ya form jiske khaane (slots) hain
Horn clause	Agar-to rule jiska conclusion sirf ek atom ho
Inference engine	Woh program jo rules laga ke naye facts nikalta hai
Inheritance	Child ko parent ki properties mil jaana
Instance	Ek asli member, jaise ali (Student frame ka)
Integrity constraint	"Yeh kabhi nahi hona chahiye" wala check
Knowledge base	Saare facts aur rules ek jagah
Negation as failure	Jo sabit na ho sake usko jhoot maan lena
Ontology	Kisi domain ki classes aur rishton ki formal list
Open world	Jo likha nahi, woh "pata nahi" hai
OWL	Web ka standard ontology format
Predicate	Property ya rishta, jaise owns(x, v)
Prolog	Logic programming language jo backward chaining karti hai
Proof tree	Goal se facts tak ka kaamyab raasta, tree ki shakal mein
Quantifier	∀ (sab ke liye) ya ∃ (koi ek to hai)
Refraction	Jo fact pehle se pata hai, usko dobara nahi banana

Word	Asaan matlab
Script	Waqiye ki sequence (gate pe aana se park karne tak)
Semantic network	Concepts ka naqsha, boxes aur arrows
Slot	Frame ka ek khaana, jaise expiryDate
SLD resolution	Prolog wala backward chaining ka tareeqa
Stratification	"Not" wale rules ko baad wali layer mein chalana
Subsumption	Ek class doosri ke andar aana (Student ⊑ Person)
TBox	DL ka woh hissa jis mein classes ke general rules hain
Tractability	Computer kitni jaldi aur pakka solve kar sakta hai
Unification	Variables ki values dhoondna taake do cheezein same ho jaayein
Variable	Koi bhi value aa sakti hai, capital letter se (X, S)